

Google Career — Day 5

Detailed To-Do & Execution Plan — Textbook Edition

Primary theme: Replace temporary in-memory persistence with a properly designed PostgreSQL persistence layer without breaking the existing API/service architecture.

Target duration: ~3 hours

Critical progression: Day 4 established FastAPI, Pydantic, API → Service → Repository, document CRUD, centralized error handling, and tests. Day 5 moves the persistence boundary from an in-memory implementation to PostgreSQL. Do not restart Day 1-4.

1. DAY 5 OBJECTIVES

By the end of Day 5, you should understand and implement:

- PostgreSQL fundamentals
- Relational databases
- Tables, rows, columns
- Primary keys
- Foreign keys
- Constraints
- INSERT
- SELECT
- UPDATE
- DELETE
- Transactions
- Index basics
- Connection management
- Database configuration
- Environment variables
- Repository → PostgreSQL transition

Project architecture should evolve from:

```
FastAPI
↓
DocumentService
↓
DocumentRepository
↓
In-Memory Dictionary
```

to:

```
FastAPI
↓
DocumentService
↓
DocumentRepository
↓
PostgreSQL
```

The API contract should remain stable.

That is an important engineering objective.

2. CURRENT REPOSITORY STATE — EVIDENCE-BASED

The current Project 1 repository is already further along than the original Day 4 baseline. It has a layered structure with `src/`, `tests/`, `docs/`, `requirements.txt`, document routes, `DocumentService`, `DocumentRepository`, Pydantic models, document data classes, custom exceptions, and centralized exception handlers.

The current architecture is documented as:

```
HTTP Client
↓
FastAPI Router
↓
DocumentService
↓
DocumentRepository
↓
In-Memory Store
```

The current API includes:

- GET /
- GET /health
- POST /documents
- GET /documents
- GET /documents/{document_id}
- PUT /documents/{document_id}
- DELETE /documents/{document_id}

The repository also documents centralized exception behavior: DocumentNotFoundError → 404, InvalidDocumentError → 400, FastAPI validation errors → 422, and unexpected exceptions → 500.

Therefore: Day 5 should not redesign the application. It should change the persistence implementation behind the existing repository boundary.

Important: actual repository state is evidence. Do not mark a feature complete merely because it appears in a plan or README; verify code, tests, and Git state.

3. DAY 5 — BLOCK 1: PostgreSQL Fundamentals

Time: 35 minutes

Do not spend 35 minutes watching a PostgreSQL tutorial. Study only the concepts that directly support today's implementation.

3.1 What is PostgreSQL?

Understand: PostgreSQL is a relational database management system that stores structured data in tables and provides SQL, transactions, constraints, indexing and concurrency mechanisms.

Your document metadata is structured:

```
documents
-----
id
name
document_type
version
status
created_at
updated_at
```

4. RELATIONAL DATABASE MENTAL MODEL

Understand:

```
Database
↓
Schema
↓
Tables
↓
Rows
↓
Columns
```

5. PRIMARY KEY

Understand why every document needs an identifier.

```
id VARCHAR(100) PRIMARY KEY
```

The primary key provides:

- uniqueness
- identity
- lookup capability
- relationship target
- indexing behavior

Interview question: Why can't we simply use the document name as the primary key? Because names may not be unique and can change.

6. CONSTRAINTS

Study NOT NULL, UNIQUE, PRIMARY KEY, and CHECK.

```
name TEXT NOT NULL
name TEXT UNIQUE
id TEXT PRIMARY KEY
CHECK (document_type IN ('requirement', 'design', 'test'))
```

Understand that constraints move some correctness guarantees into the database.

7. SQL YOU MUST KNOW TODAY

INSERT

```
INSERT INTO documents (...)  
VALUES (...);
```

SELECT

```
SELECT *  
FROM documents;
```

SELECT by ID

```
SELECT *  
FROM documents  
WHERE id = %s;
```

UPDATE

```
UPDATE documents  
SET version = %s  
WHERE id = %s;
```

DELETE

```
DELETE FROM documents  
WHERE id = %s;
```

8. TRANSACTIONS

Understand BEGIN, COMMIT, and ROLLBACK.

```
BEGIN  
operation 1  
operation 2  
COMMIT
```

versus:

```
ROLLBACK
```

A transaction allows multiple database operations to behave as one logical unit. If operation 1 succeeds but operation 2 fails, proper transaction handling can roll back the unit.

9. INDEX FUNDAMENTALS

Study the fundamentals only:

```
CREATE INDEX idx_documents_type  
ON documents(document_type);
```

An index can make reads faster, but it consumes storage and can increase write/update cost. Do not spend Day 5 designing sophisticated indexes.

10. DAY 5 — BLOCK 2: PROJECT POSTGRESQL INTEGRATION

Time: 85 minutes

This is the most important block. Keep the architecture and replace the persistence implementation.

11. FIRST — DESIGN THE TABLE

Start with fields already represented by the current API:

```
documents
-----
id
name
document_type
version
status
created_at
updated_at
```

Do not introduce fields just because they might be useful later. Only implement what today's project actually needs.

12. CREATE THE DATABASE SCHEMA

Create a reproducible database initialization or migration mechanism. Do not manually create a table once and then forget how it was created. Another developer should be able to reproduce the schema.

13. DATABASE CONFIGURATION

Do not hard-code credentials. Use environment-based configuration such as DATABASE_URL.

```
DATABASE_URL
```

Conceptually:

```
postgresql://user:password@host:5432/database
```

Actual credentials remain outside Git.

14. .ENV / SECRETS DISCIPLINE

.env may contain local development secrets and should not be committed. Provide .env.example with placeholders.

```
DATABASE_URL=postgresql://<user>:<password>@<host>:5432/<database>
```

15. DATABASE CONNECTION LAYER

Introduce a dedicated database configuration/connection boundary:

```
FastAPI
↓
Service
↓
Repository
↓
Database connection
↓
PostgreSQL
```

Understand connection lifecycle, connection reuse/pooling concept, closing resources, and failure handling. Sophisticated pooling configuration is not required today.

16. REPLACE THE IN-MEMORY REPOSITORY

Keep the conceptual DocumentRepository interface. Replace the dictionary-backed implementation with PostgreSQL.

```
Before:
DocumentRepository
↓
dict

After:
DocumentRepository
↓
PostgreSQL
```

17. CREATE DOCUMENT

Day 5 flow: POST /documents → DocumentService → DocumentRepository → INSERT → PostgreSQL. The API should continue returning the same logical response. The client should not care that storage changed.

18. GET ALL DOCUMENTS

Repository behavior is equivalent to:

```
SELECT ...
FROM documents;
```

The API remains GET /documents.

19. GET DOCUMENT BY ID

Repository behavior is equivalent to:

```
SELECT ...
FROM documents
WHERE id = ...;
```

If no row exists:

```
repository
↓
None / not found
↓
DocumentNotFoundError
↓
global exception handler
↓
HTTP 404
```

Preserve the existing error architecture.

20. UPDATE DOCUMENT

The actual repository currently documents PUT /documents/{document_id}. Do not remove or ignore PUT merely because the earlier Day 4 plan did not emphasize it. Implement PostgreSQL persistence for the update path.

Conceptually:

```
UPDATE documents
SET ...
WHERE id = ...;
```

Then verify GET returns the updated state.

21. DELETE DOCUMENT

Implement the equivalent of:

```
DELETE FROM documents
WHERE id = ...;
```

Then verify DELETE → successful response → GET same ID → 404. Use the existing successful status-code contract; do not arbitrarily change it today.

22. IMPORTANT — PRESERVE YOUR ERROR ARCHITECTURE

Routes should not regain per-route exception translation. Exceptions should bubble to centralized FastAPI handlers.

- `DocumentNotFoundError` → 404
- `InvalidDocumentError` → 400
- `RequestValidationError` → 422
- `Unexpected Exception` → 500

Do not undo centralized error handling on Day 5.

23. DATABASE TESTING

Time: 15 minutes

This is where Day 5 becomes engineering rather than 'I connected PostgreSQL.'

Test 1

Create document. Expected: POST /documents succeeds.

Test 2

Create document → GET it. Verify the same document is returned.

Test 3

Create multiple documents → GET /documents. Verify all expected records.

Test 4

Create → PUT → GET. Verify updated data.

Test 5

Create → DELETE → GET. Expected: 404.

Test 6

GET a nonexistent document. Expected: 404.

24. TEST DATABASE ISOLATION

With PostgreSQL, think about isolation:

```
Test A
↓
creates document

Test B
↓
should NOT unexpectedly see Test A's document
```

Study test database, fixture, setup, teardown, transaction rollback, and test isolation. The most sophisticated infrastructure is not required today, but the problem must be understood.

25. DAY 5 DSA — 35 MINUTES

Topic: Sliding Window Revision + Prefix Sum Introduction. Reinforce Sliding Window and introduce Prefix Sum because it recurs in subarray problems, range queries, cumulative statistics, and data-processing reasoning.

26. DSA PROBLEM 1 — Subarray Sum Equals K

Focus: Prefix Sum + Hash Map.

```
prefix[j] - prefix[i] = k
```

Therefore: $\text{sum}(i+1 \dots j) = k$. If current prefix sum is S , the previous prefix sum needed is $S - K$. This is the central insight.

27. DSA PROBLEM 2 — Product of Array Except Self

Focus: Prefix Product + Suffix Product. Understand how to avoid division using left products plus right products. Target: $O(n)$ time and $O(1)$ extra space excluding the output array, depending on the problem's space convention.

28. DSA PROBLEM 3 — Range Sum Query — Immutable

Understand $\text{prefix}[i]$ and:

```
sum(left, right) = prefix[right + 1] - prefix[left]
```

The objective is to transform repeated range queries from $O(n)$ into $O(1)$ after $O(n)$ preprocessing.

29. DSA FORMAT

- Problem
- Pattern
- Why this pattern?
- Brute Force
- Optimized Approach
- Invariant / Key Insight
- Algorithm
- Complexity
- Edge Cases
- Common Mistakes
- Alternative Approaches
- Follow-up Questions

Do not just solve the problems. You are training for Google interviews.

30. DAY 5 DATABASE INTERVIEW PREPARATION

After implementation, answer these verbally.

Fundamentals

- What is PostgreSQL?
- What is a relational database?
- What is a table?
- What is a row?
- What is a column?
- What is a primary key?
- What is a foreign key?
- What is a constraint?
- What is NOT NULL?
- What is UNIQUE?

SQL

- Explain SELECT.
- Explain INSERT.
- Explain UPDATE.
- Explain DELETE.
- What is WHERE?
- What is an index?
- Why can indexes improve reads?
- Why can indexes hurt writes?

Transactions

- What is a transaction?
- What is COMMIT?
- What is ROLLBACK?
- Why are transactions important?

Backend

- Why put PostgreSQL behind a repository?
- Why shouldn't the service execute SQL?
- How do you manage database connections?
- How do you test database-backed APIs?
- How do you isolate test data?
- What happens if PostgreSQL is unavailable?
- What happens if two requests update the same document?
- How would you scale this database?

31. DAY 5 SYSTEM DESIGN CONNECTION

Spend 10 minutes thinking about: 'How would you design the document storage service for 1 million documents?'

Use the format:

```
Requirements
↓
Scale
↓
API
↓
```

Data Model
↓
Architecture
↓
Indexes
↓
Caching
↓
Partitioning
↓
Consistency
↓
Reliability
↓
Monitoring
↓
Cost

Do not implement the scaled version. Just design it.

32. DOCUMENTATION — 10 MINUTES

Update Project 1 documentation to accurately reflect PostgreSQL persistence.

Persistence

API → Service → Repository → PostgreSQL

Also document:

- database schema
- configuration
- local setup
- migration/setup process
- test database strategy
- known limitations
- error behavior

Do not write 'Production-ready PostgreSQL architecture' unless production requirements have actually been implemented and verified.

33. GIT PLAN

Continue the meaningful Git history. Suggested sequence:

```
feat: add postgres database configuration
feat: add document database schema
refactor: replace in-memory document repository with postgres
test: add postgres document persistence tests
docs: document postgres persistence
```

Use these as suggestions, not commands to blindly copy if your actual changes differ.

34. GOOGLE INTERVIEW PREP REPOSITORY

Continue maintaining google-interview-prep. The repository already has day1/, day2/, day4/, and notes/ areas. Add Day 5 material rather than mixing it into Day 4.

Suggested structure:

```
day5/
  dsa/
    prefix_sum_notes.md
    subarray_sum_equals_k.md
    product_of_array_except_self.md
    range_sum_query.md
  databases/
    postgresql_notes.md
    sql_notes.md
    transactions_notes.md
  interview/
    day5_interview_questions.md
  README.md
```

Only create files for material you actually study/write.

35. DAY 5 — EXACT 3-HOUR EXECUTION PLAN

0:00-0:10 — Repository inspection

Pull latest changes. Check Git status. Run current tests. Read current repository structure. Confirm current API behavior.

Commands conceptually:

```
git status
git pull
python -m pytest -q
```

Output: A clean baseline.

0:10-0:45 — PostgreSQL Study

Study relational databases, tables, primary keys, constraints, SQL CRUD, transactions, indexes, and connection concepts. Output: you can explain each concept without notes.

0:45-1:05 — Database Design

Design documents with id, name, document_type, version, status, created_at, updated_at. Decide primary key, nullability, allowed document types, and timestamp behavior. Output: a documented schema.

1:05-2:10 — PostgreSQL Implementation

Implement database configuration, connection layer, schema creation/migration mechanism, PostgreSQL repository implementation, and application wiring. Keep API → Service → Repository unchanged. Verify POST, GET, GET by ID, PUT, DELETE against PostgreSQL.

2:10-2:25 — Tests

Run python -m pytest -q. Test create, list, get, update, delete, 404, validation, and test isolation.

2:25-3:00 — DSA

Solve Subarray Sum Equals K (Prefix Sum + Hash Map), Product of Array Except Self (Prefix + Suffix), and Range Sum Query — Immutable (Prefix Sum). For each: Pattern, Intuition, Brute Force, Optimized, Complexity, Edge Cases.

36. DAY 5 DEFINITION OF DONE

Do not mark Day 5 complete because you 'studied PostgreSQL.' Day 5 is complete only when the following are true.

Database

- ☐ PostgreSQL fundamentals understood
- ☐ SQL CRUD understood
- ☐ Primary key understood
- ☐ Constraints understood
- ☐ Transactions understood
- ☐ Index fundamentals understood
- ☐ Environment-based configuration understood

Project

- ☐ PostgreSQL database configured
- ☐ documents schema created reproducibly
- ☐ Repository connected to PostgreSQL
- ☐ Create works

- ☐ List works
- ☐ Get-by-ID works
- ☐ Update works
- ☐ Delete works
- ☐ Missing document still produces 404
- ☐ Existing exception architecture preserved
- ☐ API contract remains stable

Testing

- ☐ PostgreSQL-backed tests run
- ☐ Create tested
- ☐ List tested
- ☐ Get tested
- ☐ Update tested
- ☐ Delete tested
- ☐ Missing resource tested
- ☐ Validation tested
- ☐ Test isolation considered/verified

DSA

- ☐ Subarray Sum Equals K
- ☐ Product of Array Except Self
- ☐ Range Sum Query — Immutable
- ☐ Pattern recognition recorded
- ☐ Complexity understood
- ☐ Edge cases understood

Git

- ☐ Meaningful commits
- ☐ No secrets committed
- ☐ GitHub pushed
- ☐ README updated
- ☐ Actual implementation matches documentation

37. WHAT YOU MUST NOT DO ON DAY 5

- Redis
- Docker
- Kubernetes
- GCP
- React
- embeddings
- vector database
- RAG
- LangChain/LangGraph
- agents
- MLflow
- authentication
- PDF ingestion
- advanced database optimization
- microservices

You have a better engineering progression:

```
Day 1-4
Python → FastAPI → Pydantic → Service Layer → Repository → Testing → Error Handling
↓
Day 5
↓
PostgreSQL
↓
durable persistence
↓
Day 6+
↓
deeper backend/DSA
↓
ML phase
↓
RAG
↓
Agents
↓
Cloud
```

38. THE SINGLE MOST IMPORTANT DAY 5 OBJECTIVE

Make PostgreSQL a replaceable persistence implementation behind your existing DocumentRepository boundary, while keeping the API and service architecture stable.

That is much more valuable for your Google profile than simply being able to say 'I know PostgreSQL.'

Day 5 → Day 6 dependency: once PostgreSQL persistence is genuinely working and tested, move into the next backend/DSA progression without rebuilding the document service.